KRR Project — Milestone 3

From Database Schema to Ontology

Conceptual Design Explained

ANKRI Mohamed-Khalil · MICHON Charlotte · PAULIS Antoine

Academic Year 2025–2026

1. Introduction — The Core Problem

A relational database stores data efficiently in tables with rows and columns. Relationships between entities are expressed through foreign keys — one table references the primary key of another. This model works well for data storage and retrieval, but it carries a fundamental limitation: **it contains no inherent meaning**.

A column named category_id in the title_principal table tells a machine nothing about what a category is, what it represents conceptually, or how it relates to the broader domain of audiovisual media. The meaning only exists implicitly in the minds of the database designers.

An ontology solves this by making the meaning **explicit, formal, and machine-readable**. Moving from a relational schema to an OWL ontology is therefore not just a technical transformation — it is a **conceptual redesign** of how knowledge is represented.

The following sections walk through each step of this redesign for the IMDb database used in this project.

2. Step 1 — Identifying Entities: Tables to Classes

The first question to ask when looking at a relational schema is: **does this table represent a real-world concept with its own identity?** If yes, it becomes a class in the ontology. If it exists only to solve a technical constraint of the relational model (like a many-to-many junction), it does not become a class — it disappears and is replaced by an object property.

2.1 Tables that became Classes

SQL Table	Ontology Class	Reason
title	ont:Title	A film, TV show, or episode is a core real-world entity with its own identity and multiple attributes.
talent	ont:Talent	A person involved in a production is a distinct real-world individual.
genre	ont:Genre	A genre is a conceptual category used to classify titles. Kept as a class to enable reasoning and future extension.
category	ont:Category	Broad industry role classification (actor, director...). Kept as a class for the same reason as genre.
role	ont:Role	Specific credited role (e.g. camera department, composer). Kept as a class to allow traversal and querying.
content_type	ont:ContentType	The type of audiovisual work (movie, TV Episode, short...).
region	ont:Region	A geographic region associated with an alternative title.
title_type	ont:TitleType	A classification type for alternative titles (e.g. imdbDisplay, original).
title_aka	ont:AlternativeTitle	An alternative or localised name for a title. Carries its own attributes (region, language, type) so it deserves its own class.

2.2 Tables that did NOT become Classes

The following tables are pure junction tables — they exist only to express many-to-many relationships in the relational model. In the ontology they are replaced by object properties between existing classes.

SQL Table	Replaced by
title_genre	ont:hasGenre (Title → Genre)
talent_title	Covered by Participation chain (redundant)
title_aka_title_type	ont:hasType (AlternativeTitle → TitleType)
title_episode	Not mapped in this milestone (TV episode structure)

Note: The language table was mapped as an ont:Language class but no language-specific data property was declared in the OWL for this milestone.

3. Step 2 — Foreign Keys to Object Properties

Every foreign key in the SQL schema is a candidate for an **object property** in the ontology. An object property links two individuals (two class instances) together — it is the ontological equivalent of a JOIN between two tables.

The direction of the property follows the **semantic meaning** of the relationship, not necessarily the technical direction of the foreign key. For example, title_aka has a region_id foreign key pointing to region, which becomes the property isFrom going from AlternativeTitle to Region — expressing "this alternative title is from that region."

SQL Foreign Key	Object Property	Direction	Semantic meaning
title.content_type_id → content_type	ont:isClassifiedAs	Title → ContentType	A title is classified as a content type
title_genre.genre_id → genre	ont:hasGenre	Title → Genre	A title belongs to a genre
title_aka.region_id → region	ont:isFrom	AlternativeTitle → Region	An AKA title comes from a region
title_aka_title_type.title_type_id → title_type	ont:hasType	AlternativeTitle → TitleType	An AKA title has a type
title_principal (junction)	ont:contains / ont:participatesIn	Title ↔ Participation ↔ Talent	A title contains participations; a talent participates in them
talent_role.role_id → role	ont:plays	Participation → Role	A participation involves playing a role
title_principal.category_id → category	ont:as	Participation → Category	A participation is under a category

4. Step 3 — Columns to Data Properties

Columns that store **descriptive values** about an entity — strings, numbers, dates — become **data properties** in the ontology. A data property links an individual to a literal value (a plain string, an integer, a date).

The key rule is: if a column stores a value that **identifies or describes** the entity itself, it becomes a data property. If a column stores a reference to another entity (a foreign key), it becomes an object property as described in the previous section.

SQL Column	Table	Data Property	Type
primary_title	title	ont:primaryTitle	xsd:string
original_title	title	(not mapped)	—
start_year	title	ont:startYear	xsd:string
runtime_minutes	title	(not mapped)	—
talent_name	talent	ont:talentName	xsd:string
birth_year	talent	(not mapped — exists in OWL but not in R2RML)	—
genre_name	genre	ont:genreName	xsd:string
category_name	category	ont:categoryName	xsd:string
role_name	role	ont:roleName	xsd:string
content_type_name	content_type	ont:content_type_name	xsd:string
region_name	region	ont:regionName	xsd:string
title_type_name	title_type	ont:title_type_name	xsd:string
aka_title	title_aka	ont:alternativeTitleName	xsd:string
job	title_principal	ont:job	xsd:string

Note: Some columns were not mapped in this milestone either because the OWL did not declare the corresponding data property, or because the data was outside the scope of the current deliverable (e.g. runtime_minutes, original_title, birth_year).

5. Step 4 — The Reification Pattern (title_principal)

The title_principal table is the most complex table in the schema. It links a talent to a title, but also carries its own attributes: a category, a job description, a role names string, and an ordering index. It also indirectly links to the talent_role table through the talent_id.

A naive mapping would create a direct object property Talent → Title. But this would lose all the extra information attached to that specific involvement. Consider an actor who was also a producer on the same film — a direct link cannot express this.

5.1 The Problem with Direct Links

Direct approach (wrong)	Problem
ont:appearsIn : Talent → Title	Cannot attach category, role, or job to the relationship itself
One property per role: ont:actsIn, ont:directs...	Explodes the number of properties; cannot handle new roles without schema changes

5.2 The Reification Solution

The solution is to **reify the relationship** — to turn the relationship itself into a first-class individual (a node in the graph) that can carry its own properties. This is done by introducing the **Participation class**.

A Participation individual represents one specific involvement of one talent in one title. It carries:

- **ont:as** → Category (what broad role they held)
- **ont:plays** → Role (what specific role they are credited for)
- **ont:job** → a free-text job description string

The Participation individual is then connected bidirectionally:

- The Title points to it via **ont:contains**
- The Talent points to it via **ont:participatesIn**

5.3 Diagram

Node	Property	Node
ont:Title individual	ont:contains →	ont:Participation individual
ont:Talent individual	ont:participatesIn →	ont:Participation individual
ont:Participation individual	ont:as →	ont:Category individual
ont:Participation individual	ont:plays →	ont:Role individual
ont:Participation individual	ont:job →	"free-text string" (literal)

The Participation IRI is constructed from three SQL columns: title_id, talent_id, and ord. This ensures each participation is uniquely identified even when the same talent appears multiple times in the same

title with different roles.

5.4 Connecting talent_role to Participation

A complication arises from the talent_role table: it has talent_id and role_id but no title_id. This means the Participation URI cannot be constructed from talent_role alone.

The solution is an SQL join in the R2RML mapping. The MapParticipationPlaysRole TriplesMap uses an rr:sqlQuery that joins title_principal and talent_role on talent_id, allowing the Participation URI to be reconstructed and the ont:plays triple to be correctly asserted:

```
SELECT tp.title_id, tp.talent_id, tp.ord, tr.role_id
FROM title_principal tp
JOIN talent_role tr ON tp.talent_id = tr.talent_id
```

Note: This join means a talent with 3 roles in talent_role will produce 3 ont:plays triples on the same Participation individual — correctly expressing that the same participation involved multiple roles.

6. Step 5 — Why Keep Dictionary Tables as Full Classes

The small lookup tables (genre, category, role, content_type, title_type, region) could theoretically have been collapsed into plain literal values. For example, instead of ont:hasGenre pointing to a Genre individual, one could simply write a data property ont:genreName "Drama" directly on the Title.

This simpler approach was deliberately rejected for three reasons:

6.1 Enabling Graph Traversal and Querying

When Genre is a full class with individuals, you can write SPARQL queries that navigate from a genre to all titles that share it, count titles per genre, or find genres shared between two talents. With a plain literal, you can only do string matching — which is fragile and not semantically meaningful.

6.2 Enabling Future Extension

A Genre individual can receive additional properties later — for example, a description, a parent genre (for a hierarchy), or a wikidata URI. A plain literal string "Drama" cannot be extended without restructuring the entire dataset.

6.3 Making Individuals Visible in Protege

Full classes with individuals appear in Protege's "Individuals by class" browser, which is required for ontology inspection and demonstration. Plain literals do not appear there.

Approach	Queryable	Extensible	Visible in Protege
Plain literal (ont:genreName "Drama")	String match only	No	No
Full class individual (ont:Genre)	Full graph traversal	Yes	Yes

7. Step 6 — Handling Nullable Foreign Keys

Several columns in the SQL schema are defined as DEFAULT NULL — they do not always have a value. The most important example is region_id in title_aka: not every alternative title has a known region.

In an R2RML mapping, if you use a direct IRI template for a nullable column:

```
rr:objectMap [ rr:template ".../region/{region_id}" ]
```

When region_id is NULL, the mapper will either produce a broken IRI like .../region/NULL, or crash silently on that row. Both outcomes corrupt the knowledge graph.

The correct solution is to use **rr:joinCondition** instead of a direct template. A join condition causes the mapper to perform an implicit SQL join — and if the join produces no result (because the foreign key is NULL), the row is **silently and safely skipped**, producing no triple for that property on that individual.

Nullable column	Table	Solution used
region_id	title_aka	rr:joinCondition on MapRegion — NULL rows skipped
language_id	title_aka	rr:joinCondition on MapLanguage — NULL rows skipped
job	title_principal	Plain rr:column — NULL produces no triple (R2RML default behaviour)

8. Summary — Complete Mapping Reference

The following table summarises the complete conceptual transformation from the relational schema to the ontology.

Relational Concept	Ontology Equivalent	Example
Table with identity	Class	title → ont:Title
Descriptive column	Data property	primary_title → ont:primaryTitle
Foreign key	Object property	content_type_id → ont:isClassifiedAs
Simple junction table	Object property (table disappears)	title_genre → ont:hasGenre
Junction table with attributes	Reification class (new node)	title_principal → ont:Participation
Lookup / dictionary table	Class with individuals	genre → ont:Genre individuals
Nullable foreign key	Optional object property (rr:joinCondition)	region_id → OPTIONAL ont:isFrom
Multi-table join (no shared key)	SQL join in rr:sqlQuery	talent_role JOIN title_principal → ont:plays

---

End of document.